

THREADPLANE • OPEN SOURCE • ANGULAR

Threadplane



Production-ready chat, threads, and generative UI for AI agents

threadplane.ai • 2026

Contents

01 Streaming State Management

02 Thread Persistence

03 Tool-Call Rendering

04 Human Approval Flows

05 Generative UI

06 Deterministic Testing

Streaming State Management

Server-sent events don't play nicely with Angular's change detection. Zone.js patches EventSource, but the resulting microtask scheduling creates timing issues—tokens arrive faster than digest cycles complete, leading to dropped renders or, worse, accumulated state that suddenly flushes in a visual stutter. Teams typically respond by wrapping streams in `NgZone.run()`, manually calling `detectChanges()`, or building elaborate buffer-and-flush mechanisms. All of these approaches share a common failure mode: they work in development and break under production load.

The root issue isn't Zone.js itself—it's the impedance mismatch between push-based streaming and Angular's pull-based change detection model. When your LangGraph agent streams 50 tokens per second, you need state primitives that coalesce updates intelligently while remaining reactive enough to drive smooth UI. Custom solutions invariably choose wrong: either they batch too aggressively (laggy typing effect) or too little (CPU saturation from excess renders).

Signals as the Synchronization Primitive

The `agent()` function returns an Angular signals-based interface that sidesteps these problems entirely. Rather than exposing raw event streams that require manual subscription management, it provides computed signals that update atomically as tokens arrive:

```
@Component({
  selector: 'app-chat',
  template: `
    <chat-message-list [messages]="chat.messages()" />
    <chat-input (send)="chat.submit($event)" [disabled]="chat.isLoading()"
  />
  `,
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class ChatComponent {
  private readonly threadId = signal<string | undefined>(undefined);

  readonly chat = agent({
    assistantId: 'support_agent',
    threadId: this.threadId,
    onThreadId: id => this.threadId.set(id)
```

```
});  
}
```

The `messages()` signal returns `Message[]`—a runtime-neutral representation that updates as the stream progresses. Internally, the framework handles token accumulation, message boundary detection, and state reconciliation. Your component simply reads the signal; Angular's signal-based reactivity handles the rest.

The `isLoading()` signal deserves specific attention. It returns `true` from the moment you call `submit()` until the stream completes or errors. This eliminates the polling patterns teams often implement—checking message array lengths, tracking "last update" timestamps, or maintaining parallel loading flags that drift out of sync with actual stream state.

OnPush Compatibility

Signals and `OnPush` change detection are natural partners, but the pairing requires attention. When `messages()` updates, Angular marks the component dirty through signal dependencies, not through `Zone.js` event interception. This means your streaming UI actually predates `OnPush`—it *requires* it for correct behavior under load.

The production checklist question—"Are your message signals `OnPush`-compatible?"—is really asking whether your component tree properly propagates signal reads. If a parent component reads `messages()` and passes the array to a child via `@Input()`, the child must also use `OnPush` or it won't re-render when the array reference changes. The fix is straightforward: either pass the signal itself (`[messages]="chat.messages"`) or ensure `OnPush` propagates down your component tree.

Streaming state management in Angular isn't inherently difficult. It becomes difficult when you fight the framework's reactivity model instead of leveraging it. Signals provide the coalescing, the timing, and the change detection integration. Your job is to read them.

Thread Persistence

Demos work with ephemeral state. Production agents need conversation history that survives page refreshes, tab switches, and navigation—wired to LangGraph's MemorySaver backend.

Why Stateless Agent UIs Fail in Production

Every agent demo you've seen starts fresh on page load. That's fine for a conference talk. In production, users expect continuity. They start a conversation, close their laptop, resume tomorrow, and pick up where they left off. Without thread persistence, you're forcing users to re-explain context every session. Worse, you're wasting LLM tokens reconstructing state the backend already has.

LangGraph's MemorySaver stores complete conversation history server-side, keyed by thread ID. Your frontend's job is simple: remember which thread the user was talking to and reconnect on mount.

The threadId Signal and onThreadId Callback

The `agent()` function accepts a reactive `threadId` signal and an `onThreadId` callback. When `threadId` is undefined, the backend creates a new thread and fires `onThreadId` with the generated ID. Your callback persists it. On subsequent loads, you initialize the signal from storage, and the agent resumes the existing conversation.

This pattern keeps thread lifecycle management declarative. You don't manually coordinate thread creation with message sending. The agent handles it.

Persisting to localStorage

The implementation is straightforward:

```
@Component({
  selector: 'app-chat',
  template: `
    <chat [agent]="chat">
      <chat-message-list />
      <chat-input />
    </chat>
  `
})
```

```

export class ChatComponent {
  private readonly threadId = signal<string | undefined>(
    localStorage.getItem('chat_thread') ?? undefined
  );
  readonly chat = agent({
    assistantId: 'support_agent',
    threadId: this.threadId,
    onThreadId: id => {
      this.threadId.set(id);
      localStorage.setItem('chat_thread', id);
    }
  });
}

```

On first visit, `threadId` is undefined. The backend creates a thread, `onThreadId` fires, and you persist. On refresh, you read from `localStorage`, pass the existing ID, and the agent loads history from `MemorySaver`.

Thread List UI and Conversation Switching

Production apps typically need multiple conversations. A sidebar shows thread history; clicking switches context. The pattern extends naturally:

```

readonly threads = signal<string[]>(
  JSON.parse(localStorage.getItem('thread_list') ?? '[]')
);
readonly activeThreadId = signal<string | undefined>(this.threads()[0]);
readonly chat = agent({
  assistantId: 'support_agent',
  threadId: this.activeThreadId,
  onThreadId: id => {
    this.activeThreadId.set(id);
    this.threads.update(list => [id, ...list]);
    localStorage.setItem('thread_list', JSON.stringify(this.threads()));
  }
});
newConversation() {
  this.activeThreadId.set(undefined);
}
switchThread(id: string) {
  this.activeThreadId.set(id);
}

```

When `activeThreadId` changes, the agent reconnects to that thread and `messages()` reflects the restored history. No manual fetching. The reactive binding handles it.

For production, you'll likely move thread metadata to an API—titles, timestamps, archival status. The pattern remains identical: reactive signal in, persistence callback out.

Production Checklist

Before shipping, verify thread persistence end-to-end:

- **Does your agent UI resume threads correctly after a browser refresh?** Open a conversation, send messages, refresh the page. History should load automatically without user action.
- Does creating a new conversation properly clear state and generate a fresh thread ID?
- Does switching between existing threads load the correct history?
- Are thread IDs persisted before the user can navigate away?

Thread persistence is table stakes for production agent UIs. The framework gives you the primitives. Wire them correctly, and users get the continuity they expect.

Tool-Call Rendering

LangGraph agents don't just generate text—they invoke tools mid-stream, and your UI needs to reflect that execution state in real time. This means showing steps as they appear, displaying final results, and collapsing completed calls into browsable history. Getting this wrong creates a UI that feels broken during the most interesting parts of agent behavior.

The Raw Stream Problem

Tool call events arrive as discrete chunks in the SSE stream. A single tool invocation might produce five or six events: an initial call with arguments, multiple intermediate steps as the tool executes, and a final result. The raw payload includes nested metadata, partial JSON for arguments that stream incrementally, and status fields that change meaning depending on tool type.

Hand-parsing these events is fragile. You end up maintaining state machines to track which call is active, handling out-of-order delivery, and writing defensive code for malformed chunks.

Testing becomes painful because you need to simulate realistic streaming sequences. Every edge case—interrupted calls, parallel tool execution, retry logic—adds branching complexity.

The framework solves this by exposing `toolCalls()` as a normalized signal on the agent surface. Each tool call object includes its current status, accumulated steps, and final result. The stream parsing happens once, correctly, inside the transport layer.

Headless and Prebuilt Options

`@threadplane/chat` provides two components for tool call rendering. `<chat-tool-calls>` is the headless primitive—it manages the structural rendering of multiple concurrent calls but leaves visual presentation to you. `<chat-tool-call-card>` is the prebuilt option that handles common patterns: status indicators, step lists, collapsible sections, and error states.

For most production apps, start with the prebuilt card and customize from there:

```
<chat-message-list [messages]="chat.messages()">
  <ng-template #toolCalls let-calls>
    @for (call of calls; track call.id) {
      <chat-tool-call-card
        [toolCall]="call"
        [expandedByDefault]="call.status === 'running'"
      />
    }
  </ng-template>
</chat-message-list>
```



```
      (stepClick)="inspectStep($event)">
    </chat-tool-call-card>
  }
</ng-template>
</chat-message-list>
```

The card component reads status from each tool call object and adjusts its presentation accordingly. Running calls show a live step feed. Completed calls collapse to a summary with expandable history. Failed calls surface error details without disrupting the message flow.

Progressive Disclosure

Real-time tool execution benefits from progressive disclosure. Users want to see that something is happening—steps appearing as the tool runs—but they don't want permanent visual clutter once the call completes.

The `expandedByDefault` binding above handles this: calls expand while running, then collapse automatically on completion. Users can still click to expand history, but the default state keeps the conversation readable.

This pattern matters more than it seems. Agents that invoke multiple tools per response can generate substantial step output. Without automatic collapsing, the chat becomes a wall of tool metadata instead of a conversation.

Production Checklist

Before shipping, verify this behavior:

Do your tool call cards handle partial step state during streaming?

Steps arrive incrementally. A step might appear with an initial status, then update moments later with results. Your rendering logic should handle these transitions without flicker or layout shift. Test with slow network simulation to catch timing-dependent bugs that don't surface on localhost.

Human Approval Flows

Production agents that modify external state—sending emails, initiating payments, deploying infrastructure—require human oversight before execution. LangGraph provides the `interrupt()` primitive for this purpose: a mechanism that pauses graph execution at designated checkpoints and waits for explicit human authorization before proceeding.

The LangGraph Interrupt Pattern

When a LangGraph node calls `interrupt()`, execution halts and the graph emits an interrupt event containing the pending action's metadata. The graph remains suspended until it receives a `Command.RESUME` with one of three directives: proceed with the original action, proceed with modified parameters, or abort entirely. This checkpoint-based approach ensures that no consequential action executes without explicit human consent.

The challenge lies in surfacing this interrupt state to users and capturing their response without introducing fragile infrastructure. Polling-based solutions waste resources and introduce latency. Custom WebSocket implementations require maintaining connection state, handling reconnections, and synchronizing interrupt lifecycle across multiple browser tabs. Both approaches scatter interrupt logic across services, components, and connection handlers.

The `interrupt()` Signal

The `agent()` function exposes interrupt state through a dedicated signal:

```
readonly chat = agent({
  assistantId: 'deployment_agent',
  threadId: this.threadId,
  onThreadId: id => this.threadId.set(id)
});
// chat.interrupt() returns AgentInterrupt | undefined
```

When `interrupt()` returns a defined value, the agent is paused and awaiting human input. The `AgentInterrupt` object contains the action metadata emitted by the graph—typically a description of the pending operation and any parameters the user might modify. When `interrupt()` returns `undefined`, no approval is pending.

This signal-based approach eliminates the need for manual subscription management or imperative state tracking. Angular's reactivity system propagates interrupt state changes automatically, and the signal remains consistent across component re-renders.

UI Components for Interrupt Handling

The `@threadplane/chat` package provides two components for rendering interrupt flows. `<chat-interrupt-panel>` offers a prebuilt approval interface with sensible defaults. For custom designs, the headless `<chat-interrupt>` component exposes the interrupt state and action handlers without imposing markup or styling.

Both components support three user actions that map directly to resume commands:

- **Approve:** Resume execution with the original parameters
- **Edit:** Resume execution with user-modified parameters
- **Cancel:** Abort the pending action entirely

```
@Component({
  template: `
    @if (chat.interrupt(); as interrupt) {
      <chat-interrupt-panel
        [interrupt]="interrupt"
        (approve)="chat.interrupt()?.approve()"
        (edit)="chat.interrupt()?.approve($event)"
        (cancel)="chat.interrupt()?.cancel()" />
    }
  `,
})
export class DeploymentChatComponent {
  readonly chat = agent({
    assistantId: 'deployment_agent',
    threadId: this.threadId,
    onThreadId: id => this.threadId.set(id)
  });
}
```

The edit flow passes modified parameters through the `$event` payload, allowing users to adjust action details before approval. The cancel flow terminates the pending action and allows the conversation to continue without executing the interrupted operation.

Production Considerations

Interrupt flows introduce a class of edge cases that prototype implementations often ignore. Users close browser tabs. Sessions expire. Network connections drop mid-approval.

Production checklist item: *Can your agent UI recover gracefully if a user cancels an interrupt?

*

Cancellation should not leave the agent in an undefined state. The graph must handle abort commands cleanly, the UI must reflect the cancellation immediately, and subsequent user messages should resume normal conversation flow. Test this path explicitly—it executes more frequently in production than most teams anticipate.

Generative UI

Text responses hit a ceiling. When your data analysis agent returns a markdown table, users copy-paste into spreadsheets. When your booking agent describes available slots, users re-enter the same information into a form. The gap between agent output and user action creates friction that compounds across every interaction.

Production agents close this gap by emitting structured UI specifications alongside their responses. The agent doesn't return "Here are your results in a table" — it returns a render spec that becomes a live, interactive table component.

The Custom Event Pattern

LangGraph agents emit structured data through custom events during stream execution. Your agent code decides when to emit UI specifications:

```
# Agent-side: emit a render spec as a custom event
await writer.write({
  "type": "data_table",
  "columns": ["date", "amount", "category"],
  "rows": rows_so_far
})
```

On the Angular side, `@threadplane/langgraph` surfaces these through the agent's event stream. The `@threadplane/render` package consumes these specs and resolves them to Angular components at runtime.

Registry-Based Resolution

The registry pattern decouples agent output from component implementation. Your agent emits a type identifier. Your frontend maps that identifier to a component. Neither side knows implementation details of the other.

```
import { defineAngularRegistry } from '@threadplane/render';
import { DataTableComponent } from './components/data-table.component';
import { ReservationFormComponent } from './components/reservation-form.component';
import { ChartComponent } from './components/chart.component';
```

```
export const UiRegistry = defineAngularRegistry({
  data_table: DataTableComponent,
  reservation_form: ReservationFormComponent,
  chart: ChartComponent,
  // Add components without touching agent code
});
```

Components receive the spec's data through a standardized input contract. Your `DataTableComponent` receives columns and rows — it doesn't know or care that a Python agent emitted them.

Template usage is direct:

```
<render-spec [spec]="currentSpec()" [registry]="uiRegistry" />
```

Or configure the registry at the provider level with `provideRender({ registry: uiRegistry })` and omit it from individual templates.

Progressive Updates Through JSON Patch

Static specs work for complete data. Streaming scenarios require progressive updates. When your agent processes a large dataset, users shouldn't wait for completion before seeing results.

`@threadplane/render` supports JSON Patch streaming for incremental UI updates. The agent emits patches as data arrives:

```
# Initial spec
await writer.write({"type": "data_table", "columns": [...], "rows": []})
# Patches as rows arrive
for row in process_rows():
    await writer.write({"op": "add", "path": "/rows/-", "value": row})
```

The frontend applies patches to the live spec. Rows appear as they're processed. Charts animate as data points arrive. Users see progress, not loading spinners.

The Decoupling Advantage

Tight coupling between agent and frontend creates deployment dependencies. Changing a table column requires coordinated releases. Adding a new visualization blocks on frontend implementation.

The registry pattern inverts this. Agents emit specs against a stable contract. Frontend teams add components independently. You can ship a new heatmap type in your registry without

redeploying agents — they'll use it when ready.

This also enables A/B testing component implementations, graceful degradation for unknown types, and environment-specific registries (richer components in desktop, simplified in mobile).

Production checkpoint: Can your agent emit UI components without tight coupling to the frontend codebase? If adding a new visualization requires changes to both agent and frontend in lockstep, the integration is too brittle for production iteration speed.

Deterministic Testing

Agent UIs are notoriously difficult to test. Every call to a live LLM introduces variability—different token sequences, timing variations, occasional model updates that subtly change output format. The result is flaky tests, slow CI pipelines, and an inability to reproduce the exact edge case a user reported. Teams ship agent features with low confidence because their test suites can't verify behavior deterministically.

Why Live LLM Testing Fails

Testing against real LLM APIs introduces three fundamental problems. First, response content varies between runs. The same prompt might yield slightly different phrasing, breaking snapshot tests or exact-match assertions. Second, latency compounds. A single agent interaction might take 2-5 seconds; a test suite with 50 agent tests becomes a 4-minute bottleneck. Third, you can't manufacture edge cases on demand. How do you test interrupt handling if the model decides not to request human input? How do you verify your tool call UI when the model skips the tool entirely?

Deterministic testing requires control over the event stream itself.

MockAgentTransport: Scripted Event Sequences

MockAgentTransport replaces the network layer entirely. You provide a scripted sequence of events, and the transport emits them on demand. No server, no network, no variability.

This approach lets you test streaming behavior by controlling exactly when each token arrives. You can simulate interrupts at precise moments, inject tool calls with specific payloads, and verify error handling by emitting failure events. Your tests become reproducible scenarios rather than probabilistic hopes.

mockLangGraphAgent(): Writable Signal Control

For component-level testing, mockLangGraphAgent() provides an even more direct approach. It returns an agent surface where every signal is writable—you set the state, and your component reacts.

```
describe('ChatComponent', () => {  
  it('displays interrupt panel when interrupt is pending', () => {  
    const agent = mockLangGraphAgent();
```



```

const fixture = TestBed.createComponent(ChatComponent);
fixture.componentRef.setInput('agent', agent);

agent.interrupt.set({
  value: { question: 'Confirm deletion?' },
  options: ['confirm', 'cancel']
});
fixture.detectChanges();

expect(fixture.nativeElement.querySelector('chat-interrupt-
panel')).toBeTruthy();
expect(fixture.nativeElement.textContent).toContain('Confirm
deletion?');
});
});

```

This pattern isolates component behavior from streaming mechanics. You're testing how your UI responds to state—not whether the transport correctly parses SSE frames.

Testing in Isolation

Each agent capability becomes independently testable. For streaming, set `isLoading` to true and progressively update messages to verify your typing indicators and incremental rendering. For tool calls, populate `toolCalls` with specific payloads and assert your `ChatToolCallCardComponent` renders the expected UI. For generative UI via `render-spec`, test your registered components against static specs without involving the agent layer at all.

Interrupts deserve particular attention. Set `interrupt` to various payloads and verify your `ChatInterruptPanelComponent` handles each type—multiple choice, free text, confirmation dialogs. Call the `resume` function and assert `interrupt` clears correctly.

Production Checklist

Before shipping agent features, verify this: **Do your agent component tests run offline and complete in under 100ms each?**

If not, you're either hitting real APIs or your test setup carries unnecessary overhead. Deterministic agent testing should feel like testing any other Angular component—fast, reliable, and completely under your control.